

WealthLink

Single View of Wealth Platform

Entity-resolution proof of concept for a multi-subsidiary banking group

Technical & Commercial Reference

How the demo works, why it was built this way, and how to present and defend it

Who this document is for

Sales & presales teams get a plain-English walkthrough of the problem, the demo, and the pitch.

Engineers & solution architects get a module-by-module technical breakdown, the entity-resolution methodology, the data model, the API surface, and a prepared FAQ for technical due diligence.

Look for the **orange** boxes for business framing and the **blue** boxes for engineering depth.

Contents

1	Executive Summary	3
2	The Business Problem	5
2.1	Why subsidiaries can't see the whole customer	5
2.2	The cost of not solving it	5
2.3	Why this is hard, not just tedious	6
3	How the Solution Works, in Plain English	7
3.1	The pipeline at a glance	7
3.2	Stage 1: Simulating the problem	7
3.3	Stage 2: Resolving identity (entity resolution)	8
3.4	Stage 3: Building the golden Single View of Wealth	8
3.5	Stage 4: The dashboard — what you actually see in a demo	8
3.6	The five-minute live demo script	9
4	Architecture Overview	10
4.1	Technology stack	10
4.2	Module map	10
4.3	Why these particular technology choices	11
5	Technical Deep Dive: Synthetic Data Generation	12
5.1	Purpose and reproducibility	12
5.2	Population and scale	12
5.3	The noise model: simulating five imperfect subsidiary systems	13
5.3.1	Worked example: <code>_vary_first_name</code>	13
5.4	Statistical realism: salaries, balances and mortgages	14
5.5	The unified <code>records</code> table	14
6	Technical Deep Dive: Entity Resolution Engine	16
6.1	What is probabilistic record linkage?	16
6.2	Configuration choices, end to end	16
6.2.1	Link type: <code>dedupe_only</code>	16
6.2.2	Comparisons: one per identity field	16
6.2.3	Blocking rules: making the problem computationally tractable	17
6.2.4	Training: a two-stage probability estimate	17
6.2.5	Prediction, clustering, and the match threshold	18
6.3	Turning pairwise scores into a per-record confidence	18
6.4	Results on the current seeded dataset	19
7	Technical Deep Dive: Wealth Aggregation	20
7.1	Building the golden profile	20
7.2	Choosing a display name from noisy variants	20
7.3	Wealth tiers and percentile scoring	21
7.4	The “Match Explanation”: field-by-field agreement	21
7.5	Search, dashboard summary, and data-quality queries	21

8	Technical Deep Dive: API & Application Layer	22
8.1	Startup auto-bootstrap	22
8.2	models.py vs. schemas.py	22
8.3	Endpoint reference	22
9	Technical Deep Dive: Authentication & Access Control	24
9.1	The login flow	24
9.2	Roles	24
10	Technical Deep Dive: Exports	26
11	Technical Deep Dive: The Dashboard Frontend	27
11.1	Design philosophy	27
11.2	Client-side authentication	27
11.3	The four views, and what drives them	27
12	The Command-Line Demo Script	29
13	Data Model Reference	30
14	Frequently Asked Technical Questions	31
15	Known Simplifications & Path to Production	33
16	Glossary	34

Chapter 1

Executive Summary

Single View of Wealth (SVW) is a working demonstration of how a banking group with several subsidiaries can stop treating each subsidiary’s customer data as a separate island, and instead see *one consolidated picture of wealth* for every individual across the whole group — automatically, and without asking any subsidiary to change its own systems first.

The proof of concept (PoC) does three things end-to-end, live, in front of a viewer:

1. **Simulates the real problem.** It generates a realistic synthetic population of 10,000 people and “leaks” them across five fictional subsidiary banks as 25,000+ payroll records and tens of thousands of banking-product holdings (current accounts, savings, investments, mortgages) — each subsidiary capturing the same person slightly differently (nicknames, typos, formatting differences, missing fields), exactly the way real organically-grown subsidiary IT estates do.
2. **Resolves identity.** It runs an industry-grade probabilistic record-linkage engine (**Splink**, the open-source entity-resolution library originally built by the UK Ministry of Justice) over every one of those noisy records and works out, statistically, which records belong to the same real person — without ever being told the answer.
3. **Builds the golden record.** It aggregates every resolved person’s salary, cash, savings, investments and mortgage balances into a single “Single View of Wealth” profile, complete with a wealth tier, a percentile ranking, and a transparent, field-by-field explanation of *why* the system believes these records belong to the same person.

All of this is wrapped in a polished web dashboard and a documented REST API, so it can be demoed live, explored by a technical evaluator, or scripted end-to-end from the command line (`demo.py`) with no server at all.

BUSINESS TAKEAWAY

Why a buyer should care. Most banking groups grew by acquisition or by spinning up subsidiaries with their own core banking systems. None of them share a customer ID. The result: the group cannot answer simple questions like “*what is our total relationship with this household?*”, cannot cross-sell effectively, under-prices risk, and struggles with KYC/AML obligations that require knowing who their customers really are across the whole group. This PoC proves, on realistic synthetic data, that the hard part — resolving identity across messy, independently-captured records — is solvable today, with mature open-source technology, in well under a minute, on a single laptop.

ENGINEERING NOTE

What’s genuinely real here vs. simulated. The data is 100% synthetic (Faker-generated, seeded for reproducibility) — nothing here is a real customer. What *is* real is the entity-resolution methodology: Splink’s probabilistic (Fellegi–Sunter) matching model, trained with Expectation-Maximisation, scored and clustered exactly as it would be against production data. The PoC also keeps a hidden “ground truth” (which records really belong to which synthetic person) purely so it can measure its own precision/recall — a luxury that disappears

in production, where this same pipeline would run unchanged but unverified by ground truth.

Chapter 2

The Business Problem

2.1 Why subsidiaries can't see the whole customer

Picture a banking group with five subsidiaries: a retail bank, a wealth manager, a trust company, a commercial bank, and a private bank. Each was built or bought at a different time, on different core banking software, with its own employee/payroll system and its own customer-onboarding process. None of them were ever designed to talk to each other, and none of them share a common customer or employee identifier.

When the same person – say, a senior employee who also banks personally with two of the group's subsidiaries – shows up in all five systems, each system has its own slightly different snapshot of who they are:

- One spells their name in full (*Jonathan*), another uses a nickname (*Jon*) or just an initial (*J*).
- One stores their address as “10 Main Street”, another abbreviates it to “10 Main St”.
- One has their postcode with a space, another without.
- Email addresses differ by format (*jon.smith@gmail.com* VS. *jsmith23@gmail.com*) because each subsidiary's onboarding form generated its own.
- Fields are sometimes simply missing – a phone number never captured, an address left blank.

BUSINESS TAKEAWAY

None of this is a data-entry failure to be blamed on any one team. It is the *normal, expected* state of any organisation that grew through multiple systems. The problem only becomes visible – and costly – when the group tries to answer a question that spans subsidiaries: “how much does this person really hold with us, across everything?”

2.2 The cost of not solving it

- **Missed commercial opportunity.** A private banking subsidiary cannot identify that one of its high-net-worth clients is also a payroll-earning senior employee at the group's commercial bank – a natural cross-sell and relationship-pricing opportunity that simply never surfaces.
- **Mispriced risk.** Credit and exposure decisions made subsidiary-by-subsidiary underestimate a customer's true indebtedness or overstate their true liquidity, because no single view of their mortgage, savings and investment position exists.
- **Regulatory and KYC/AML exposure.** “Know Your Customer” and anti-money-laundering obligations increasingly expect a group-wide view of a customer relationship, not five fragmented ones. Failing to produce one is a genuine compliance and audit finding waiting to happen.

- **Poor customer experience.** Customers experience the group as five unconnected companies, repeating the same information five times, with no acknowledgement that the bank already knows them elsewhere in the group.

2.3 Why this is hard, not just tedious

The obvious-sounding fix – “just match the records where the names and addresses are the same” – breaks immediately on real data, because real data never matches *exactly*. “Jon Smith, 10 Main St” and “Jonathan Smith, 10 Main Street” are clearly the same person to a human reader, but an exact-match SQL JOIN sees two completely unrelated strings. Scaling that judgement call to hundreds of thousands of records, consistently and auditably, is what this PoC exists to prove is achievable.

BUSINESS TAKEAWAY

This is the industry problem usually called building a **Single Customer View** or a **golden record**. It is a well-trodden, well-understood data-management discipline – this PoC demonstrates it can be stood up quickly, cheaply, and transparently, using mature open-source tooling rather than a multi-year, multi-million-pound master-data-management programme.

Chapter 3

How the Solution Works, in Plain English

3.1 The pipeline at a glance

The platform is a four-stage pipeline. Figure 3.1 shows the flow; each stage is a Python module in the codebase, named for what it does.

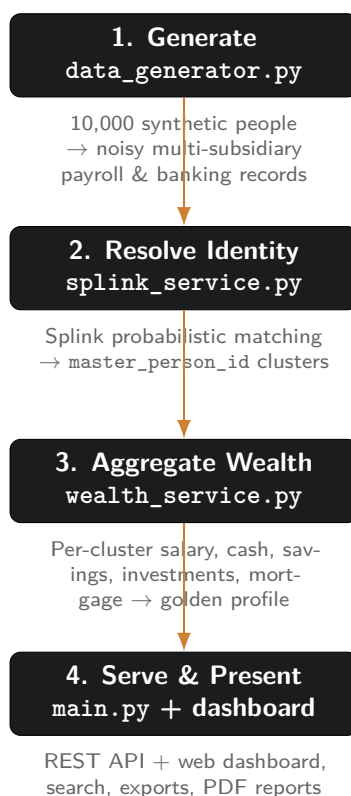


Figure 3.1: The end-to-end SVW pipeline. Every stage persists its output to a single embedded DuckDB database file (`data/svow.duckdb`), so each stage can be re-run independently.

3.2 Stage 1: Simulating the problem

Rather than risk (or need) real customer data, the PoC generates its own realistic population using **Faker**, a library that produces plausible-looking but entirely fictional names, addresses, emails and dates. It then simulates, in software, exactly the kind of imperfect data capture described in Chapter 2: nicknames, abbreviated addresses, reformatted phone numbers, missing fields, and so on. Every run uses the same random seed, so the same demo always produces the same (impressive) results.

3.3 Stage 2: Resolving identity (entity resolution)

This is the heart of the product. Think of it like a very careful detective comparing thousands of slightly different witness statements and working out which ones describe the same person. The system looks at every pair of records that could plausibly be the same person and asks, field by field: *do the names sound alike? Is the date of birth close? Does the postcode match? Does the email share a username?* It then combines all of these clues into a single confidence score between 0% and 100%.

Records whose confidence score clears a chosen bar are linked together into a group representing one real person, given a new shared ID (`master_person_id`, e.g. MP00001). This is exactly what a human analyst would do manually, given unlimited time – the system just does it for tens of thousands of records in under a minute, consistently, and shows its working.

BUSINESS TAKEAWAY

This is the single most valuable and most defensible part of the demo. It is powered by **Splink**, an open-source entity-resolution engine originally developed by the UK Ministry of Justice and now widely used across UK government and industry. It is not a black box or a toy fuzzy-matching script – it is the same statistical methodology (Fellegi–Sunter probabilistic record linkage) used in production data-linkage systems handling tens of millions of records.

3.4 Stage 3: Building the golden Single View of Wealth

Once the system knows which records belong to the same person, it combines their financial information into one profile:

- **Salary** is averaged across that person’s linked payroll records (each subsidiary’s payroll feed is recording the same job, so averaging smooths small reporting differences rather than double-counting).
- **Cash, savings, investments and mortgage balances** are summed, because someone can genuinely hold several real accounts of the same type across different subsidiaries, and each one is real wealth or real debt.
- **Net wealth** is cash + savings + investments minus mortgage debt.

Every resolved person is then ranked against everyone else in the bank’s population and assigned a wealth percentile and a simple commercial segment label (Mass Market, Affluent, High Net Worth, Ultra High Net Worth, or Negative Equity) – the kind of segmentation a private bank or wealth manager already uses to decide who gets a relationship manager call.

3.5 Stage 4: The dashboard — what you actually see in a demo

The dashboard (Figure 3.2, described below) is what gets shown in a sales meeting. It has four views:

Dashboard Headline KPIs (population, total records, resolved clusters, duplicates found, average match confidence, total net wealth under management), and the centrepiece: an **“Entity Resolution In Action”** panel that shows one real resolved person exactly as their records looked *before* the system ran (several records that look like different people) and *after* (one consolidated profile) – pulled live from whatever the current run produced, not a canned screenshot.

Directory A searchable list of every resolved person, with the option to export the whole list as CSV.

Profile A single person’s full dossier: their wealth breakdown, every linked source record with its own confidence score, and a “Match Explanation” panel that says, in plain language, exactly which fields agreed across the linked records and which ones still disagree (e.g. two different postcodes were seen). This is what makes the system’s reasoning auditable rather than a black box.

Data Quality A transparency page: a histogram of how confident the system was across the whole population, a breakdown of cluster sizes, and a “manual review queue” of the handful of lowest-confidence matches a real bank’s analyst team would want to sanity-check by hand.

BUSINESS TAKEAWAY

The demo’s best moment is the Dashboard’s before/after panel: showing a viewer several records that visibly look like different people (different name spellings, different contact details) and then showing the single consolidated profile the system produced from them, with a confidence score attached to every link. It turns an abstract claim (“we do entity resolution”) into a concrete, visual, two-second “aha”.



Figure 3.2: The four views of the WealthLink dashboard (`app/static/`), all backed live by the JSON API – nothing on screen is hardcoded.

3.6 The five-minute live demo script

The repository includes `demo.py`, a self-contained command-line script with no server required, designed to be run live:

1. Generates the synthetic dataset.
2. Picks one person who has noisy records across at least three subsidiaries and prints them – visibly looking like different people.
3. Runs the full Splink pipeline.
4. Prints the resolved `master_person_id`, every subsidiary that was linked to it, and each link’s confidence score.
5. Prints that person’s final Single View of Wealth: salary, cash, savings, investments, mortgage, and net wealth.

BUSINESS TAKEAWAY

This script is the fastest way to demonstrate the core value proposition to a non-technical stakeholder without touching a browser: run one command, watch “five different people” become “one person, with a complete financial picture”, in well under a minute.

Chapter 4

Architecture Overview

4.1 Technology stack

Component	Role
FastAPI	Python web framework serving the REST API and the static dashboard.
DuckDB	Embedded, file-based analytical database (<code>data/svow.duckdb</code>); no separate database server to install or operate.
Splink (DuckDB backend)	Probabilistic record-linkage / entity-resolution engine – the core IP of the demo.
Faker	Synthetic data generator (names, addresses, emails, phone numbers, dates).
Pandas / NumPy	In-memory data manipulation between generation, linkage and persistence steps.
PyJWT + bcrypt	JSON Web Token issuing/verification and password hashing for the login layer.
ReportLab	Server-side PDF rendering for the profile export report.
Tailwind CSS (CDN) + Chart.js (CDN)	Dependency-free, build-step-free dashboard frontend (plain HTML/JS).

Table 4.1: Full technology stack. Everything runs in a single process with no external services – deliberately, so the whole PoC is one `uvicorn` command.

ENGINEERING NOTE

Every dependency is pinned to a version range in `app/requirements.txt`. There are no calls to external APIs or cloud services anywhere in the codebase – the entire pipeline, from synthetic data generation to PDF export, runs fully offline.

4.2 Module map

Module	Responsibility
<code>data_generator.py</code>	Seeded synthetic data generation: people, noisy multi-subsidiary payroll records, and noisy banking-product records, all persisted into one unified <code>records</code> table.
<code>models.py</code>	Internal domain models (Pydantic) describing each DuckDB table's row shape.
<code>splink_service.py</code>	Splink configuration and the train → predict → cluster pipeline.
<code>wealth_service.py</code>	Aggregates clusters into wealth profiles; powers search, dashboard and data-quality queries.
<code>auth.py</code>	JWT login, password hashing, and role-based access control.

Module	Responsibility
<code>exports.py</code>	CSV and PDF report rendering on top of <code>wealth_service</code> 's existing queries.
<code>schemas.py</code>	Public API request/response contracts (Pydantic), independent of the internal storage shape.
<code>main.py</code>	FastAPI app wiring, startup auto-bootstrap, and every HTTP endpoint.
<code>static/index.html</code> , <code>static/app.js</code>	The dependency-free dashboard frontend, served at <code>/</code> .
<code>demo.py</code>	Standalone before/after CLI walkthrough; no server required.

Table 4.2: Module responsibilities. See Chapters 5–12 for a deep dive on each.

4.3 Why these particular technology choices

Why DuckDB, not Postgres/MySQL?

DuckDB is an embedded, file-based, single-process analytical database – there is nothing to install, configure, or operate, and it is extremely fast for the bulk SQL aggregation this PoC does. `splink_service.py` also uses Splink's native `DuckDBAPI` backend, so the entity-resolution engine and the storage layer share the same technology, keeping the whole stack to one moving part.

Why Splink specifically?

Splink implements the Fellegi–Sunter probabilistic record-linkage model with a modern, well-tested API, purpose-built comparison functions for names/dates/emails/postcodes, and a scalable in-process execution engine (DuckDB, Spark, or Athena backends). It is open-source, actively maintained, and already proven at scale in government and industry deployments – a far stronger foundation than a bespoke fuzzy-matching script.

Why a build-step-free frontend?

The dashboard is plain HTML/JavaScript using Tailwind CSS and Chart.js from a CDN, with no bundler, framework, or `npm install` step. This keeps the entire PoC runnable from a single `uvicorn` command, which matters when the audience is a prospect's laptop in a meeting room, not a CI/CD pipeline.

Chapter 5

Technical Deep Dive: Synthetic Data Generation

Module: `app/data_generator.py`

5.1 Purpose and reproducibility

Because this is a PoC and no real customer data is available (or should be used), `data_generator.py` builds an entirely synthetic but statistically realistic population, and then simulates how five independent subsidiary IT systems would *imperfectly* capture that same population. Every random draw is seeded (`SEED = 777`), so the exact same dataset — and therefore the exact same demo — is produced on every machine, every time.

ENGINEERING NOTE

Reproducibility is achieved with a dedicated `random.Random(seed)` instance threaded through every generation function (never the global `random` module), plus `Faker.seed(seed)` for the Faker-driven fields. This means two calls to `generate_all()` with the same seed are bit-for-bit identical, which is what lets the threshold-sweeping precision/recall analysis in Chapter 6 be trusted.

5.2 Population and scale

- **10,000** ground-truth people (`N_PEOPLE`), each independently generated by Faker's `en_GB` locale (UK-style names, addresses, postcodes, phone numbers), aged 21–66.
- Each person is independently recorded by between 1 and 4 of the group's five subsidiaries' payroll systems, with an *exact* target distribution (not just approximate): 2,000 people recorded by 1 subsidiary, 3,000 by 2, 3,000 by 3, and 2,000 by 4 – totalling exactly **25,000** payroll records.
- Each person is also independently assigned a banking-product “bundle” (payroll-only, payroll + deposits, payroll + investments, payroll + mortgage, or all products), which determines which of the four banking-product types (current account, savings, investment, mortgage) they hold, at which subsidiaries, and how many accounts of each.

The five subsidiaries are modelled as a Python enum (`app/models.py`) with both a display name and a short internal code:

Listing 5.1: The five subsidiaries (`app/models.py`)

```
1 class Subsidiary(str, Enum):
2     A = "Hawksworth Retail Bank"
3     B = "Calder Wealth Partners"
4     C = "Brightfield Trust"
5     D = "Sterling Commercial Bank"
6     E = "Ridgeway Private Bank"
```

5.3 The noise model: simulating five imperfect subsidiary systems

The same seven identity fields (first name, last name, date of birth, email, phone, address, postcode – city is left untouched) are independently “re-captured” for every record, payroll or banking product alike, via one shared function, `_noisy_identity_capture()`. This guarantees the noise model is identical regardless of which kind of record is being generated, so Splink genuinely cannot use record type as a shortcut.

Field	Noise simulated
First name	55% unchanged; then a real nickname if one exists for that name (e.g. <i>William</i> → <i>Will/Bill/Billy</i>); then an initial only (<i>W</i>); then a random upper/lower-case variant.
Last name	85% unchanged; then an upper/lower-case variant; then (rarely) a transposed-letter typo simulating a transcription error.
Date of birth	95% unchanged; 5% has day and month transposed (a classic data-entry error, only when the day is ≤ 12 so it remains a valid date).
Address	Street-type tokens (<i>Street, Road, Avenue, ...</i>) independently abbreviated (<i>St, Rd, Ave, ...</i>) with 50% probability per occurrence.
Postcode	30% chance the internal space is stripped (<i>TF2R 2LQ</i> → <i>TF2R2LQ</i>).
Email	50% chance of reusing the canonical address verbatim; otherwise regenerated under a different formatting convention (dot/no-dot/initial/underscore, different webmail domain, optional numeric suffix) – simulating a different subsidiary’s own account-issuing system.
Phone	50% chance of reuse verbatim; otherwise formatting characters (spaces, brackets, dashes) are stripped.
Missing values	Independently, email is nulled 12% of the time, phone 12%, address 10% – fields simply never captured by that subsidiary.

Table 5.1: The full identity-noise model, applied identically to every payroll and banking-product record.

BUSINESS TAKEAWAY

These aren’t arbitrary numbers – they encode the exact failure modes a real bank’s data-quality team will recognise immediately: nicknames, abbreviations, formatting drift, and missing fields. Showing a prospect this table is often more convincing than the demo itself, because it proves the PoC was built to mirror their actual data problem, not a sanitised toy version of it.

5.3.1 Worked example: `_vary_first_name`

Listing 5.2: Cumulative-threshold noise sampling (`data_generator.py`)

```

1 def _vary_first_name(first_name: str, rng: random.Random) -> str:
2     key = first_name.lower()
3     roll = rng.random()
4     if roll < FIRST_NAME_UNCHANGED_PROB:           # 0.55
5         return first_name
6     if key in NICKNAMES and roll < FIRST_NAME_NICKNAME_PROB: # 0.80
7         return rng.choice(NICKNAMES[key])
8     if roll < FIRST_NAME_INITIAL_PROB:             # 0.92
9         return first_name[0]
10    return first_name.upper() if rng.random() < 0.5 else first_name.lower()

```

ENGINEERING NOTE

Note the thresholds are *cumulative*, not independent per-branch probabilities – a single `roll` is drawn once and compared against ascending thresholds in sequence. This is a common source of bugs if a future edit inserts a new branch without re-deriving the thresholds below it; the module-level constants are deliberately named for the branch they gate (e.g. `*_UNCHANGED_PROB`) so the intent is harder to invert by accident.

5.4 Statistical realism: salaries, balances and mortgages

A flat `uniform(low, high)` salary or balance would look obviously synthetic – real income and wealth distributions are right-skewed, with most people sitting well below the maximum and a shrinking population stretching out toward it. The generator therefore uses a **shifted lognormal distribution**:

Listing 5.3: Shifted lognormal sampler (`data_generator.py`)

```
1 def _lognormal(rng, floor, median_above_floor, sigma, high):
2     """floor + a lognormal-distributed excess over it, capped at high."""
3     return min(high, floor + rng.lognormvariate(
4         math.log(median_above_floor), sigma))
```

ENGINEERING NOTE

Shifting (adding the lognormal excess *on top of* a floor) rather than simply clamping a plain lognormal at the floor matters: clamping would pile up a visible spike of values sitting exactly on the floor in a histogram, which looks obviously synthetic. Shifting makes the floor smoothly unreachable, tapering down toward it instead – indistinguishable from a real income/wealth histogram at a glance.

This same function (with different parameters) drives annual salary (floor £30,000, capped at £250,000), current-account balances (floor £50, capped at £50,000), savings (floor £200, capped at £200,000), and investments (floor £500, capped at £500,000). Bonuses use a **Beta(2,5)** distribution (skewed toward modest payouts, capped at 50% of salary), and mortgage balances use a **triangular distribution** centred on a 3.5×-salary affordability multiple (realistic UK mortgage-lending behaviour), clamped to £50,000–£600,000.

BUSINESS TAKEAWAY

Net wealth is deliberately defined as cash + savings + investments – mortgage, with no offsetting property-value asset. This means a customer with only payroll income and a mortgage will show a *negative* net wealth figure on the dashboard – this is by design (matching the agreed wealth formula), not a bug, and is worth pre-empting in a demo before a sharp-eyed viewer asks about it.

5.5 The unified records table

Every kind of record – payroll *and* all four banking-product types – is written into one table, distinguished by a `record_type` column (`PAYROLL`, `CURRENT_ACCOUNT`, `SAVINGS_ACCOUNT`, `INVESTMENT`, `MORTGAGE`). Payroll-only columns (`employee_id`, `annual_salary`, `bonus`) and product-only columns (`account_id`, `balance`) are simply left `NULL` on rows where they don't apply.

ENGINEERING NOTE

This unification is a deliberate, documented design decision (see `app/README.md`): banking products are *not* attached to a person via a clean, trusted, ground-truth index. They carry exactly the same noisy identity-capture fields as payroll records and must be resolved by the same Splink pipeline. This closes what was originally a simplification in an earlier version of the PoC, and means a banking product's attachment to a person is subject to the same genuine linkage risk a payroll record always has been – which is the technically honest way to model how a real banking group's systems actually behave.

Chapter 6

Technical Deep Dive: Entity Resolution Engine

Module: `app/splink_service.py`

This chapter is the one most likely to be probed in technical due diligence. It explains *what Splink is*, *how it is configured here*, and *why* each configuration choice was made.

6.1 What is probabilistic record linkage?

Splink implements the **Fellegi–Sunter model**, the foundational statistical framework for record linkage (Fellegi & Sunter, 1969). Instead of asking “do these two records match exactly?”, it asks, for every comparable field: *given that two records really are the same person, how likely is it that this field would look the way it does (the **m probability**)?* And *given that two records are **not** the same person, how likely is it that this field would agree by pure chance (the **u probability**)?* The ratio of these two probabilities, summed (in log form) across every field, produces a single overall match-probability score for the pair.

BUSINESS TAKEAWAY

In plain terms: the system has learned, from the data itself, things like “two unrelated people sharing the same postcode happens fairly often (it’s just a neighbourhood), so postcode agreement alone is weak evidence – but two unrelated people sharing the same email address is essentially impossible, so email agreement is very strong evidence.” It weighs every clue by how surprising it would be if the two records were *not* the same person.

6.2 Configuration choices, end to end

6.2.1 Link type: dedupe_only

Every noisy record – payroll and banking-product alike, from all five subsidiaries – lives in one single pool to be deduplicated against itself, rather than two separate datasets to be linked together. `_load_linkage_pool()` reads every row of the unified `records` table, projected down to just the columns Splink needs: the unique ID plus the seven noisy identity-comparison columns. Notably, `record_type` itself, and payload columns like `employee_id/account_id`, are deliberately excluded from what Splink sees – they are semantically incomparable across record types and would mislead the comparison model rather than help it.

6.2.2 Comparisons: one per identity field

Listing 6.1: Splink settings (`splink_service.py`, `_settings()`)

```
1 comparisons=[
2     cl.NameComparison("first_name"),
3     cl.NameComparison("last_name").configure(term_frequency_adjustments=True),
4     cl.DateOfBirthComparison("date_of_birth", input_is_string=True,
5                             datetime_format="%Y-%m-%d"),
6     cl.EmailComparison("email"),
```

```

7  cl.LevenshteinAtThresholds("phone", [1, 2]),
8  cl.LevenshteinAtThresholds("address", [2, 5, 10]),
9  cl.PostcodeComparison("postcode").configure(term_frequency_adjustments=True),
10 ],

```

Field	Why this comparison
First/last name	<code>NameComparison</code> layers exact match, Jaro–Winkler similarity (catches typos/case differences) and a phonetic (double-metaphone) fallback – exactly what is needed for “Jon” vs. “Jonathan” or a transposed-letter surname. Surname additionally gets term-frequency adjustment, so a common surname (e.g. Smith) is treated as weaker evidence than a rare one.
Date of birth	Parses the ISO date and scores by calendar distance, so a day/month transcription error still scores far higher than an unrelated date of birth.
Email	Understands the <code>username@domain</code> structure and rewards a matching username even when the domain differs (or vice-versa) – the “john.smith@gmail.com” vs. “johnsmith@gmail.com” pattern.
Phone, address	Both are free-text, formatting-sensitive fields. Levenshtein (character edit-distance) thresholds are a cheap, robust similarity signal for punctuation and abbreviation differences.
Postcode	<code>PostcodeComparison</code> with term-frequency adjustment: postcodes are strong identity signals, but the adjustment stops common postcode districts from over-crediting matches between unrelated people who simply live in the same area.

Table 6.1: Field-by-field comparison rationale.

6.2.3 Blocking rules: making the problem computationally tractable

ENGINEERING NOTE

Comparing every record against every other record is an $O(n^2)$ problem – roughly 49,000 records would mean over a billion candidate pairs. **Blocking** restricts full comparison to pairs that share at least one plausible signal of being the same person, and Splink takes the *union* of every blocking rule’s candidate pairs, so a pair only needs to satisfy *one* rule to be scored.

Listing 6.2: Blocking rules (`splink_service.py`)

```

1  blocking_rules_to_generate_predictions=[
2      block_on("first_name", "last_name"),
3      block_on("email"),
4      block_on("phone"),
5      block_on("date_of_birth", "postcode"),
6      block_on("last_name", "postcode"),
7      block_on("substr(first_name,1,1)", "last_name", "date_of_birth"),
8  ],

```

Each rule captures a different realistic “this pair is plausibly the same person” signal: a shared email, a shared phone, a matching full name, a matching DOB+postcode, a matching surname+postcode, or a matching first-initial+surname+DOB (to catch “J Smith” style records where the first name was reduced to an initial).

6.2.4 Training: a two-stage probability estimate

1. **Seed the prior.** A small set of high-precision deterministic rules (e.g. “emails are identical and non-null”, or “phone, surname *and* the rest agree”) give Splink a sensible starting

estimate of how rare a random match is overall. None of these rules alone decides a final match – they only seed the prior.

2. **Learn the *u* probabilities.** `estimate_u_using_random_sampling` samples random pairs of records to learn how often each field agrees *purely by chance* between unrelated people.
3. **Learn the *m* probabilities via Expectation-Maximisation.** `estimate_parameters_using_expectation_maximisation` is run three times, against three *different* blocking rules (name+surname, then date-of-birth, then postcode). This is Splink’s standard identifiability pattern: every comparison must be trained on data it was not itself blocked on, otherwise the model would be circularly validating itself.

Listing 6.3: Training pipeline (`splink_service.py`, `_build_and_train_linker()`)

```

1 linker.training.estimate_probability_two_random_records_match(
2     _deterministic_rules(), recall=0.6)
3 linker.training.estimate_u_using_random_sampling(max_pairs=2e6, seed=EM_SEED)
4 linker.training.estimate_parameters_using_expectation_maximisation(
5     block_on("first_name", "last_name"))
6 linker.training.estimate_parameters_using_expectation_maximisation(
7     block_on("date_of_birth"))
8 linker.training.estimate_parameters_using_expectation_maximisation(
9     block_on("postcode"))

```

6.2.5 Prediction, clustering, and the match threshold

Once trained, the model scores every candidate pair surfaced by blocking (`linker.inference.predict`), keeping only pairs scoring at least 5% (purely to keep the intermediate predictions table a manageable size). It then runs **connected-components clustering**: two records end up in the same final cluster if and only if they are linked, directly or transitively, by a chain of pairwise scores at or above the **cluster match threshold** – a separate, higher bar of **0.75 (75%)**.

ENGINEERING NOTE

`CLUSTER_MATCH_THRESHOLD = 0.75` was chosen empirically, by sweeping the threshold from 0.3 to 0.95 and measuring pairwise precision and recall against the data generator’s own hidden ground truth (a luxury only possible because this is a synthetic demo). Thresholds between 0.5 and 0.9 all produced statistically identical, near-optimal results (precision ~100%, recall ~98%); 0.75 sits comfortably in the middle of that stable plateau, giving headroom in both directions before results degrade. This sweep predates banking products joining the same linkage pool as payroll records – the noise model is unchanged, but the absolute precision/recall figures should be treated as historical (payroll-only) until the sweep is re-run against the larger, blended pool.

6.3 Turning pairwise scores into a per-record confidence

Splink scores *pairs* of records, not individual records – so “how confident is the system in this one record’s link?” is not a native Splink output. `_per_record_match_probability()` derives it as the strongest pairwise edge connecting that record to another member of its own final cluster:

Listing 6.4: Deriving a per-record confidence score (`splink_service.py`)

```

1 def _per_record_match_probability(clusters_df, predictions_df):
2     cluster_of = clusters_df.set_index(UNIQUE_ID_COL)["cluster_id"]
3     within_cluster = predictions_df[
4         cluster_of.reindex(predictions_df[f"{UNIQUE_ID_COL}_l"]).values
5         == cluster_of.reindex(predictions_df[f"{UNIQUE_ID_COL}_r"]).values
6     ]
7     best_l = within_cluster.groupby(f"{UNIQUE_ID_COL}_l")["match_probability"].max()

```

```

8 best_r = within_cluster.groupby(f"{UNIQUE_ID_COL}_r")["match_probability"].max()
9 best = pd.concat([best_l, best_r]).groupby(level=0).max()
10 return clusters_df[UNIQUE_ID_COL].map(best).fillna(1.0)

```

A record that ended up alone in a singleton cluster (no duplicate found at all) defaults to 100% confidence – there is no second record to be uncertain against, so there is no ambiguity about which person it is.

6.4 Results on the current seeded dataset

Metric	Value
Payroll records	~25,000
Banking-product records	~24,000
Total noisy records resolved together	~49,000
Resolved clusters (unique people)	~10,150
Duplicates found (records – clusters)	~38,900
Average per-record match confidence	~100%
Pairwise precision (payroll-only sweep, historical)	~100%
Pairwise recall (payroll-only sweep, historical)	~98%

Table 6.2: Reproducible results from the seeded dataset (**SEED** = 777). Precision/recall were measured against the generator’s hidden ground truth, only possible in this synthetic demo.

BUSINESS TAKEAWAY

Precision means: of the records the system says belong together, how many actually do?
Recall means: of the records that actually belong together, how many did the system successfully find? Near-100% precision with ~98% recall means the system is extremely trustworthy when it does link two records (very few false merges), while still catching the overwhelming majority of true duplicates – exactly the trade-off a bank should want: it is far worse to wrongly merge two different customers’ financial data than to occasionally leave two records of the same person unlinked (which the manual review queue exists to catch).

Chapter 7

Technical Deep Dive: Wealth Aggregation

Module: *app/wealth_service.py*

7.1 Building the golden profile

Once `clusters` exists, `build_wealth_profiles()` runs a single SQL statement that rebuilds the `wealth_profiles` table from scratch, joining every cluster against the unified `records` table and aggregating by `record_type`.

Listing 7.1: Aggregation logic (abbreviated; full statement in `wealth_service.py`)

```
1 salary_agg AS (  
2     SELECT c.master_person_id, AVG(r.annual_salary) AS salary  
3     FROM clusters c JOIN records r USING (source_record_id)  
4     WHERE r.record_type = 'PAYROLL' GROUP BY 1  
5 ),  
6 cash_agg AS (  
7     SELECT c.master_person_id, SUM(r.balance) AS cash  
8     FROM clusters c JOIN records r USING (source_record_id)  
9     WHERE r.record_type = 'CURRENT_ACCOUNT' GROUP BY 1  
10 )  
11 -- ...savings_agg, investments_agg, mortgage_agg follow the same SUM pattern  
12 SELECT bn.master_person_id, bn.name,  
13     COALESCE(sal.salary, 0) AS salary,  
14     COALESCE(cash.cash, 0) AS cash,  
15     COALESCE(sav.savings, 0) AS savings,  
16     COALESCE(inv.investments, 0) AS investments,  
17     COALESCE(mtg.mortgage, 0) AS mortgage,  
18     COALESCE(cash.cash,0)+COALESCE(sav.savings,0)+COALESCE(inv.investments,0)  
19     - COALESCE(mtg.mortgage,0) AS net_wealth  
20 FROM best_name bn  
21 LEFT JOIN salary_agg sal USING (master_person_id)  
22 -- ...remaining LEFT JOINS
```

ENGINEERING NOTE

Average, not sum, for salary: duplicate payroll records represent the *same* underlying job, captured slightly differently by each subsidiary's feed (rounding, timing, minor re-entry) – summing would double- or quadruple-count one salary. **Sum, not average, for balances:** a person can genuinely hold several real current accounts, savings pots, or mortgages across different subsidiaries, and each one is real wealth or real debt that must be added, not averaged away.

7.2 Choosing a display name from noisy variants

A cluster of linked records rarely agrees on exactly how to spell the person's name. `ranked_names` picks the “best” variant to display, in priority order: prefer a fuller name over an initial (“Billy” over “B”), then prefer proper-case over ALL-CAPS or all-lowercase, then prefer whichever variant was recorded most often across the cluster's records.

7.3 Wealth tiers and percentile scoring

The Profile page’s wealth score is a true **percentile rank** of `net_wealth` computed across every resolved profile in the current run (`PERCENT_RANK() OVER (ORDER BY net_wealth)`), and the wealth tier is a simple threshold ladder on top of it:

Tier	Net wealth threshold
Negative Equity	< £0
Mass Market	< £100,000
Affluent	< £500,000
High Net Worth	< £2,000,000
Ultra High Net Worth	≥ £2,000,000

Table 7.1: Wealth tiers – a simplified version of the proposition segments real banks use for relationship-manager targeting.

7.4 The “Match Explanation”: field-by-field agreement

For each of six identity fields (email, phone, address, postcode, date of birth, last name), `get_profile_detail()` checks whether *every* linked record – payroll and banking-product alike – agreed on a single non-null value. Where they disagree, every distinct value seen is surfaced (e.g. “TF2R 2LQ” vs. “TF2R2LQ”):

Listing 7.2: Field agreement (`wealth_service.py`, `get_profile_detail()`)

```

1 profile["field_agreement"] = [
2     {
3         "field": field,
4         "is_consistent": len({r[field] for r in all_records if r[field]}) <= 1,
5         "distinct_values": sorted({r[field] for r in all_records if r[field]}),
6     }
7     for field in _FIELD_AGREEMENT_COLUMNS
8 ]

```

BUSINESS TAKEAWAY

This is the feature that turns “trust our AI” into “see exactly what our system saw”. Every claim the Profile page makes is computed live from the underlying linked records – there is no canned narrative anywhere in this panel. For a regulated industry, this kind of field-level explainability is frequently a hard requirement, not a nice-to-have, and is worth leading with in a technical evaluation.

7.5 Search, dashboard summary, and data-quality queries

`search_person()` matches against *both* a profile’s chosen display name and every underlying linked record’s name, so a search for “Jonathan Smith” still finds a cluster whose display name resolved to the more common “John Smith” variant. `get_dashboard_summary()` and `get_quality_metrics()` compute every KPI, chart and the manual-review queue directly from `clusters/records/wealth_profiles` with no ground truth involved – meaning these exact same queries would work unmodified against real production data.

Chapter 8

Technical Deep Dive: API & Application Layer

Modules: *app/main.py*, *app/schemas.py*, *app/models.py*

8.1 Startup auto-bootstrap

On first launch, FastAPI's `lifespan` hook (`_bootstrap_demo_data()`) seeds the two demo user accounts, and – only if the database doesn't already contain them – generates the synthetic dataset, runs the Splink pipeline, and builds wealth profiles, in that order. Each step is individually idempotent (checked via `has_generated_data()` / `has_run_linkage()` / `has_wealth_profiles()`), so restarting the app never silently redoes ~30 seconds of work, and deleting `data/svow.duckdb` is all that's needed to force a clean rebuild.

8.2 `models.py` vs. `schemas.py`

ENGINEERING NOTE

The codebase deliberately keeps two parallel sets of Pydantic models. `models.py` describes the internal DuckDB row shapes (what is actually stored); `schemas.py` describes the public API contract (what `/docs` promises callers). This means the OpenAPI schema can evolve independently of the storage layer – e.g. a column could be renamed internally without necessarily being a breaking API change, or vice versa.

8.3 Endpoint reference

Method	Path	Auth	Description
POST	<code>/auth/login</code>	–	Exchange username/password for a bearer token.
GET	<code>/auth/me</code>	any	The authenticated user's username and role.
POST	<code>/generate-data</code>	admin	(Re)generate the synthetic dataset.
POST	<code>/run-linkage</code>	admin	Run the Splink pipeline and rebuild wealth profiles.
GET	<code>/wealth/{id}</code>	any	Golden wealth profile for one resolved person.
GET	<code>/wealth/{id}/detail</code>	any	Full dossier: linked records, percentile/tier, match explanation.
GET	<code>/wealth/{id}/export/pdf</code>	any	The dossier rendered as a downloadable PDF.
GET	<code>/search?q=</code>	any	Search profiles by name; empty <code>q</code> lists all, alphabetically.
GET	<code>/export/directory.csv</code>	any	Every matching profile as CSV, uncapped.

Method	Path	Auth	Description
GET	/dashboard	any	Platform-wide summary metrics.
GET	/dashboard/showcase	any	One representative profile for the before/after panel.
GET	/quality	any	Confidence histogram, cluster-size distribution, review queue.
GET	/export/review-queue.csv	any	The full manual-review backlog as CSV.
GET	/health	–	Liveness/readiness probe.
GET	/	–	The dashboard frontend.

Table 8.1: Full API surface. Interactive schemas with field-level descriptions live at `/docs` (Swagger) and `/redoc`.

BUSINESS TAKEAWAY

The fact that every dashboard feature is backed by a documented, independently-callable REST API is itself a selling point: a prospect’s own engineering team could integrate this entity-resolution pipeline into their existing systems via the API, without ever touching the bundled dashboard.

Chapter 9

Technical Deep Dive: Authentication & Access Control

Module: `app/auth.py`

9.1 The login flow

A minimal JWT (JSON Web Token) login sits in front of every data endpoint. `POST /auth/login` verifies a username/password against a `users` table (passwords hashed with **bcrypt**, never stored in plain text) and, on success, issues a signed JWT containing the username, role, and a 60-minute expiry.

Listing 9.1: Token issuing and verification (`auth.py`)

```
1 def create_access_token(username: str, role: str) -> str:
2     expires_at = datetime.now(timezone.utc) + timedelta(minutes=
3         ACCESS_TOKEN_EXPIRE_MINUTES)
4     payload = {"sub": username, "role": role, "exp": expires_at}
5     return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)
6
7 def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
8     payload = decode_access_token(token) # raises on invalid/expired
9     user = _get_user(payload.get("sub")) # re-fetched live, not trusted from
10         the token
11     if user is None:
12         raise credentials_error
13     return user
```

ENGINEERING NOTE

`get_current_user` deliberately re-fetches the user row from the database rather than trusting the role embedded in the token alone. This means a role change (or account deletion) takes effect on the *next* request, rather than waiting up to 60 minutes for the old token to expire – a small but meaningful correctness choice, made cheap here because every other lookup in the app already opens a fresh, short-lived DuckDB connection per request anyway.

9.2 Roles

Role	Can do
admin	Everything an analyst can, plus <code>POST /generate-data</code> and <code>POST /run-linkage</code> (the expensive, destructive pipeline operations).
analyst	Search, view profiles, dashboard, and exports (read-only on the resolved data).

Table 9.1: The two seeded demo roles, enforced via `Depends(require_role(...))` on the FastAPI route definitions themselves.

Two demo accounts are seeded automatically on first startup: `admin / admin123` and `analyst / analyst123`.

BUSINESS TAKEAWAY

Role-based access control isn't just a login screen – it maps directly onto a real operational concern: in a production deployment, *running* the entity-resolution pipeline (which rebuilds the entire customer view) should be a privileged, audited action, while *reading* resolved profiles for day-to-day relationship management should be available to a much broader analyst population. This PoC already models that separation.

ENGINEERING NOTE

POC-scoped, by design, not by oversight. The JWT signing secret falls back to a logged, clearly-named insecure development default (`svw_jwt_secret` environment variable overrides it); there is no signup, password reset, or refresh-token flow; and the frontend stores the bearer token in browser `localStorage` rather than an `httpOnly` cookie. All of this is adequate for a local demo and explicitly called out as needing hardening before any exposure beyond local/demo use – see Chapter 15.

Chapter 10

Technical Deep Dive: Exports

Module: `app/exports.py`

`exports.py` is a pure rendering layer: every function takes data already produced by `wealth_service` and turns it into a downloadable file, with no new database queries beyond calling the existing lookups with a much larger limit than their on-screen defaults (so an export never silently truncates what it claims to cover).

- **GET `/export/directory.csv`** – every resolved profile matching the search query (or all profiles) as CSV, uncapped at up to 100,000 rows.
- **GET `/export/review-queue.csv`** – the full manual-review backlog (default top 500 lowest-confidence clusters, not just the dashboard widget's top 10).
- **GET `/wealth/{id}/export/pdf`** – a single profile's full dossier rendered as a polished PDF via **ReportLab**: KPI summary table, linked subsidiary records, banking products by subsidiary, and the field-level match explanation table.

BUSINESS TAKEAWAY

The PDF export is a tangible, leave-behind artifact for a sales meeting: a single resolved customer's full Single View of Wealth, with its supporting evidence (every linked record and its confidence score), as a document a relationship manager could genuinely attach to a case file.

Chapter 11

Technical Deep Dive: The Dashboard Frontend

Modules: `app/static/index.html`, `app/static/app.js`

11.1 Design philosophy

The frontend is intentionally framework-free: plain HTML and vanilla JavaScript, styled with Tailwind CSS and charted with Chart.js, both loaded from a CDN with no bundler or build step. The entire dashboard is two files served as static assets by FastAPI's `StaticFiles` mount. This keeps the whole PoC runnable from one `uvicorn` command – there is no separate frontend build, deploy, or version-compatibility story to manage during a demo.

11.2 Client-side authentication

A login overlay gates the whole single-page app. On successful `POST /auth/login`, the returned bearer token is stored in `localStorage` and attached as an `Authorization: Bearer <token>` header to every subsequent API call (the `api()` helper in `app.js`). The admin-only “Generate Data”/“Run Linkage” button pair is hidden client-side for the `analyst` role (the server-side `require_role("admin")` dependency is the actual enforcement; the UI hiding is purely a UX nicety, not a security boundary).

11.3 The four views, and what drives them

View	Backed by
Dashboard	<code>GET /dashboard</code> (KPIs) + <code>GET /dashboard/showcase</code> (before/after panel) + chart data derived from the same payload.
Directory	<code>GET /search</code> (empty <code>q</code> browses all profiles alphabetically); “Export CSV” calls <code>GET /export/directory.csv</code> .
Profile	<code>GET /wealth/{id}/detail</code> ; “Export Linked Data” (client-side JSON download), “Export Profile Report” (<code>GET /wealth/{id}/export/pdf</code>), and “Copy Master ID”.
Data Quality	<code>GET /quality</code> ; “Export CSV” calls <code>GET /export/review-queue.csv</code> (the full backlog, not just the on-screen top 10).

Table 11.1: Every dashboard view is a thin rendering layer over the JSON API – nothing shown is hardcoded or mocked.

ENGINEERING NOTE

Worth knowing for a live Q&A: the “Entity Resolution In Action” before/after panel and the Profile page’s record list both render *whatever the live API returns*, including on a freshly regenerated dataset. If a prospect asks to regenerate the data live and re-run linkage during

the demo (the admin-only buttons support exactly this), the dashboard will reflect the new run's actual results, not a cached or scripted outcome.

Chapter 12

The Command-Line Demo Script

Module: *demo.py*

For situations where a live browser demo isn't practical (a terminal-only environment, a quick sanity check, or a presenter who prefers a scripted narrative), `demo.py` reproduces the entire “before/after Splink” story from the command line, with no server required:

1. **Generates** the synthetic dataset (identical to what the API's `/generate-data` does).
2. **Selects a compelling example:** a person recorded by at least 3 subsidiaries, holding every banking-product type, with at least 2 different first-name spellings across their records – chosen specifically so the “before” state is visually convincing (it looks unmistakably like several different people).
3. **Prints the “before” state:** every payroll and banking-product record for that person, exactly as a subsidiary's own system would show it – different name spellings, different contact details.
4. **Runs the full Splink pipeline** and rebuilds wealth profiles.
5. **Prints the “after” state:** the resolved `master_person_id`, every subsidiary linked to it with its own confidence score, and the consolidated banking-product holdings.
6. **Prints the Single View of Wealth:** salary, cash, savings, investments, mortgage, and net wealth.
7. **Prints the platform-wide dashboard summary** as a closing summary of the whole run.

ENGINEERING NOTE

The person-selection query uses `EXISTS` subqueries rather than `INNER JOINS` against the unified `records` table specifically to avoid a Cartesian-product blow-up: a person can hold multiple rows of the same product type, so joining directly would inflate `COUNT(*)` with rows that have nothing to do with the number of distinct *payroll* records being counted.

BUSINESS TAKEAWAY

Run it with: `python demo.py` (or, using the bundled environment on Windows, `.\env\python.exe demo.py`). It takes well under a minute and needs nothing else running – ideal as a “let me just show you” moment in a meeting where spinning up a browser and a server isn't worth the friction.

Chapter 13

Data Model Reference

All persistent state lives in a single DuckDB file, `data/svow.duckdb`, opened with a fresh short-lived connection per request (DuckDB supports a single writer at a time; this trades a small amount of per-call overhead for a much simpler concurrency story, appropriate for a demo).

Table	Contents
<code>persons</code>	Ground-truth only: <code>person_index</code> , name, date of birth, email, phone, address, city, postcode. Exists <i>only</i> because this is a synthetic demo – a real bank would never have this table; it only ever sees <code>records</code> . Used purely to power the before/after narrative and to measure linkage precision/recall against ground truth.
<code>records</code>	~49,000 noisy rows – one unified table for every kind of subsidiary record. Columns: <code>source_record_id</code> , <code>person_index</code> (hidden ground-truth foreign key), <code>subsidiary</code> , <code>record_type</code> , identity fields (name/contact, with noise/nulls), plus whichever payload columns apply (<code>employee_id</code> / <code>annual_salary</code> / <code>bonus</code> for payroll, <code>account_id</code> / <code>balance</code> for the four product types), <code>currency</code> .
<code>clusters</code>	Splink's output: <code>source_record_id</code> , <code>master_person_id</code> , <code>match_probability</code> – covering every row of <code>records</code> regardless of <code>record_type</code> .
<code>wealth_profiles</code>	<code>master_person_id</code> , name, salary, cash, savings, investments, mortgage, <code>net_wealth</code> – the golden, aggregated profile per resolved person.
<code>users</code>	<code>username</code> , <code>password_hash</code> , role, <code>created_at</code> – login accounts (Chapter 9).

Table 13.1: DuckDB schema. `persons` and `person_index` are the only tables/columns that would not exist in a production deployment.

ENGINEERING NOTE

A person may hold zero, one, or several rows of a given banking-product type, each independently tagged with the subsidiary it sits at and independently noised – reflecting how a real banking group's customers genuinely do scatter products across subsidiary banks (a mortgage at one, savings at another), each captured imperfectly by that subsidiary's own system.

Chapter 14

Frequently Asked Technical Questions

This chapter anticipates the questions a technically literate prospect, auditor, or internal stakeholder is likely to ask, so a presenter doesn't have to improvise an answer live.

Q: Is any of this real customer data?

No. Every person, address, email and financial figure is generated by Faker and the statistical models in `data_generator.py` (Chapter 5). The dataset is fully synthetic and reproducible (fixed random seed), and contains no real individuals.

Q: Why Splink, rather than a simpler fuzzy-matching script (e.g. Levenshtein distance + a

A single similarity score across one field is exactly the naive approach that fails on real data: it cannot weigh *which* fields matter more, cannot account for how common or rare a given value is (a shared common surname is weak evidence; a shared rare email is strong evidence), and has no statistically principled way to combine multiple fields' evidence into one score. Splink's Fellegi-Sunter model learns those weights from the data itself (Chapter 6), and is a mature, widely-deployed open-source library rather than bespoke, unvalidated logic.

Q: How do you stop the system merging two different people into one (a false merge)?

Two layers of protection: (1) the trained model weighs disagreeing evidence heavily – two records with a different date of birth or a structurally different email score very low, even if names happen to match; (2) the **cluster match threshold** (0.75) was empirically chosen by sweeping thresholds against ground truth and selecting a value in the middle of a wide, stable plateau where precision was measured at ~100% – i.e. essentially zero false merges on this dataset. In production, the manual review queue (Chapter 7) is the safety net for the lowest-confidence multi-record clusters a model should not be trusted blindly on.

Q: What about the records that *should* have been linked but weren't (recall)?

Measured at ~98% on the historical payroll-only sweep – the small remainder are cases where the noise on a record was severe enough (e.g. initial-only first name *and* a heavily abbreviated address *and* a missing email) that even a careful human reviewer would likely also miss the connection without additional context. These are exactly the records that should, and do, surface in the manual review queue.

Q: How would this scale to millions of records in production?

Splink supports Spark and Amazon Athena backends in addition to the in-process DuckDB backend used here, specifically for horizontal scale-out; none of the comparison logic, blocking rules, or training methodology in `splink_service.py` would need to change to move backends. The DuckDB backend used in this PoC comfortably handles tens of thousands of records in well under a minute; the blocking-rule strategy (Chapter 6) is exactly the lever used to keep larger datasets computationally tractable.

Q: Why DuckDB instead of a “real” database like Postgres?

DuckDB is a genuine, widely-used analytical database engine (not a toy) – it’s embedded and file-based purely so the PoC has zero infrastructure to stand up. For a production deployment, the SQL in `wealth_service.py` is portable, standard SQL and would migrate to Postgres/Snowflake/BigQuery with minimal changes; the single-writer constraint called out in Chapter 15 is the main thing that would need to change for a multi-user production workload.

Q: Is this auditable? Can you explain why two records were linked?

Yes – this is one of the platform’s deliberate design priorities. The Profile page’s “Match Explanation” panel (Chapter 7) shows, field by field, exactly which identity attributes agreed across every linked record and which still disagree, computed live from the actual data with no canned narrative. Every linked record also carries its own per-record confidence score (Chapter 6), and the Data Quality page surfaces the lowest-confidence clusters for manual review.

Q: What’s not production-ready yet?

See Chapter 15 for the full list – in short: the JWT secret/login flow needs production-grade secret management and a refresh-token story; DuckDB’s single-writer model needs revisiting for concurrent multi-user write access; and the precision/recall figures need re-validating against the current blended (payroll + banking-product) linkage pool rather than the historical payroll-only sweep.

Q: How is the demo guaranteed to look the same every time?

Every random choice in both `data_generator.py` and `splink_service.py` is seeded (population generation, noise injection, and Splink’s own random-sampling and EM training all take an explicit seed). `POST /generate-data` followed by `POST /run-linkage` produces bit-for-bit identical results on every run, on every machine.

Q: Does the system know it’s looking at synthetic data with a hidden answer key?

No – and this is an important distinction to make confidently in due diligence. `splink_service.py` only ever reads the seven noisy identity columns from `records`; it never sees `person_index` (the hidden ground-truth link) or `persons` at all. The `persons` table and `person_index` column exist solely so *the demo itself* can show a true before/after comparison and so this PoC’s own developers could measure precision/recall during development – a production deployment would never have that table, and the resolution pipeline runs identically either way.

Chapter 15

Known Simplifications & Path to Production

Every item below is a deliberate, documented PoC-scope decision (see `README.md`'s "Notes & known simplifications"), not an oversight – being able to state this distinction confidently is itself part of being well-prepared for technical questions.

- **Single currency.** All amounts are generated in GBP only; a `currency` column exists for schema completeness, but no FX conversion is implemented.
- **Net wealth excludes property value.** Net wealth is cash + savings + investments – mortgage; it does not model property value as an offsetting asset. A "payroll + mortgage only" customer therefore necessarily shows negative net wealth *by design*, not as a bug – worth pre-empting before a viewer asks.
- **Banking-product noise reuses the payroll noise model.** Product records reuse the exact same nickname/abbreviation/typo/missing-value functions as payroll, rather than modelling product-specific data-quality patterns (e.g. a bank's own onboarding form might have different failure modes than its payroll feed) separately.
- **Precision/recall figures are historical.** The ~100%/~98% pairwise precision/recall figures were measured by sweeping the cluster threshold against ground truth back when the linkage pool was payroll-only; that sweep has not been re-run since banking products joined the same pool, and there is no in-repo tooling to re-run it beyond the one-off development exercise that produced the original figures.
- **DuckDB is single-writer, opened per request.** Appropriate for a demo or low-concurrency internal tool; a production, multi-analyst deployment would need a database that supports concurrent writers (e.g. Postgres), or a write-serialising service layer in front of DuckDB.
- **Authentication is POC-scoped.** Two seeded demo accounts, a 60-minute JWT with no refresh flow, no signup/password-reset/user-management UI, and a development-only JWT signing-secret fallback (clearly logged as insecure) if `svw_jwt_secret` is not set. The bearer token lives in browser `localStorage`, not an `httpOnly` cookie. All adequate for a local demo; all would need hardening before exposure beyond local/demo use.
- **Framework-free frontend.** A deliberate trade-off for demo portability (Chapter 11); a production UI would likely warrant a proper frontend build pipeline, component framework, and design system.

BUSINESS TAKEAWAY

None of these are reasons to hesitate on the core claim of the demo – that entity resolution across messy multi-subsidiary data is solved, today, with mature tooling. They are the honest, scoped list of engineering work between *this PoC* and a *production deployment*, which is exactly the list a buyer's own technical team will want to see exists and is owned.

Chapter 16

Glossary

Blocking rule A rule used to restrict which pairs of records are even considered for comparison, to keep an otherwise quadratic comparison problem computationally tractable.

Cluster / clustering The step where pairwise match probabilities are turned into groups: records connected (directly or transitively) by sufficiently confident links end up in the same cluster, representing one resolved real-world person.

Deterministic rule A simple, high-precision exact-match rule (e.g. “emails are identical”) used only to seed Splink’s prior estimate of how rare a random match is – not a final decision rule.

Entity resolution The general discipline of determining which records, across one or more datasets, refer to the same real-world entity (here, the same person).

Expectation-Maximisation (EM) An iterative statistical algorithm used to estimate the model’s match/non-match probabilities for each comparison from unlabelled data.

Fellegi–Sunter model The classical statistical framework for probabilistic record linkage that Splink implements, weighing field-level agreement/disagreement by how likely it is under a “same person” vs. “different person” hypothesis.

Golden record / Single Customer View The single, consolidated, trusted profile of an entity (here, a person’s wealth) built by resolving and aggregating multiple source records.

m probability The probability that a given field would agree the way it does, *given that* the two records being compared really are the same person.

Master person ID The stable, human-friendly ID (e.g. MP000001) assigned to a resolved Splink cluster, representing one real individual.

Match probability Splink’s pairwise score (0–1) that two specific records describe the same entity; this PoC also derives a per-*record* confidence (Chapter 6) from the strongest pairwise edge within its cluster.

Probabilistic record linkage Record linkage performed by combining the statistical likelihood of agreement across multiple fields, rather than requiring an exact match.

u probability The probability that a given field would agree the way it does *by pure chance*, given that the two records being compared are *not* the same person.